



Department of Electrical and Computer
Engineering

Instructor : Hassan Tfaily

Lab 1 : Introduction to the MC9S12DT256 microcontroller

Introduction and objectives

In this week's lab you will be introduced to the MCU project board P BMCUSLK , discover the main features of the MC9S12DT256 version of the HCS12D microcontrollers , with a quick overview on the main topics that we will work on during this lab .

You will start , as well , writing an assembly program using the CodeWarrior software and testing some programs' functionalities with their corresponding addressing modes.

Pre Lab1 :

MC9S12DT256 main features :

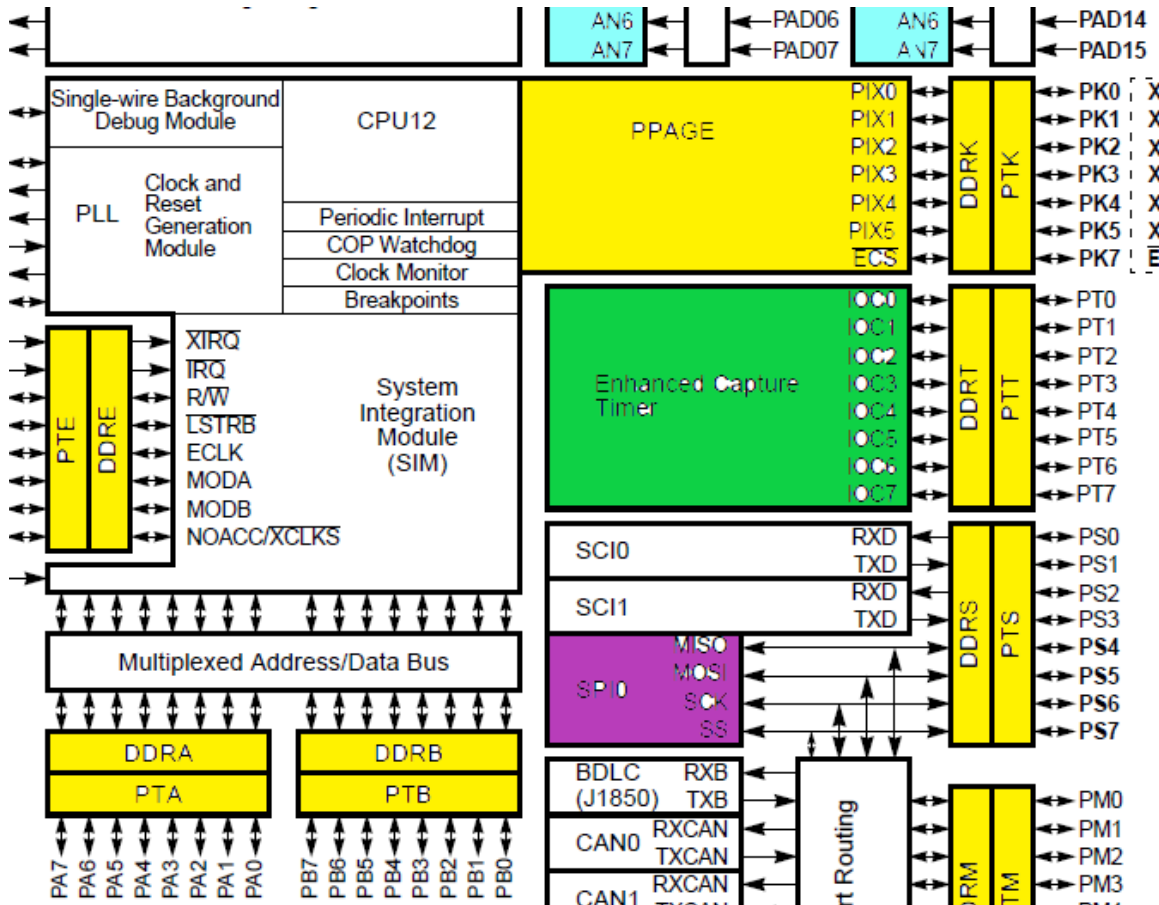
MC9S12DT256 MCU is a 16-bit device composed of standard on-chip peripherals including :

- a) 16-bit central processing unit (HCS12 CPU), 256K bytes of Flash EEPROM, 12K bytes of RAM, 4K bytes of EEPROM.
- b) **8 bits PORTS (with their Data Direction Port)**
- c) 2 timers: TCCNT, MCCNT, triggering interrupts when they overflow. MCCNT is also used to create PERIODIC DELAYS.

d) 3 Serial Peripheral Interface SPI

e) **ATD converter**: 2 ATD MODULES , each one has 8 analog inputs , 8 digital OUTPUTS that can be displayed on 8 or 10 bits .

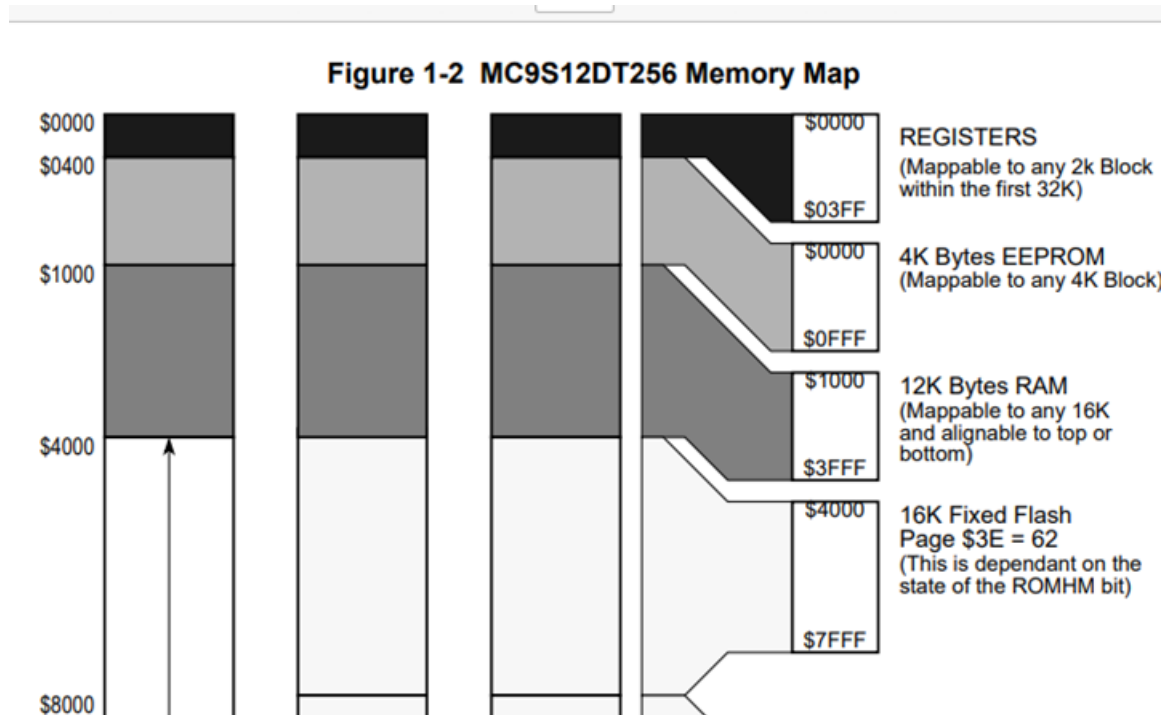
f) **PWM module**: 8 PWM Channels used to generate a periodic signal with a programmable period and duty cycle.



I. Memory mapping :

- The registers block is reserved for the GPIO (Data registers as PORTA, Data direction registers as DDRA, Control registers, Status registers) starting at @0000.
- The declared variables in the program are saved in the volatile RAM starting at @ \$1000.

- c) The instructions of the program are saved in the non-volatile Flash starting from @\$4000.



II. CPU accumulators and registers

- CPU registers are different than the GPIO , they are used by the ALU to supply the operands, they do not have addresses in the memory, while the GPIO have addresses between \$0000 and \$03FF.
- X / Y / SP are 16 bits registers that point to addresses in Data memory.
- D is a combination of A and B.
- PC is a 16 bits register that points to the next instruction to be executed in the Flash program memory .

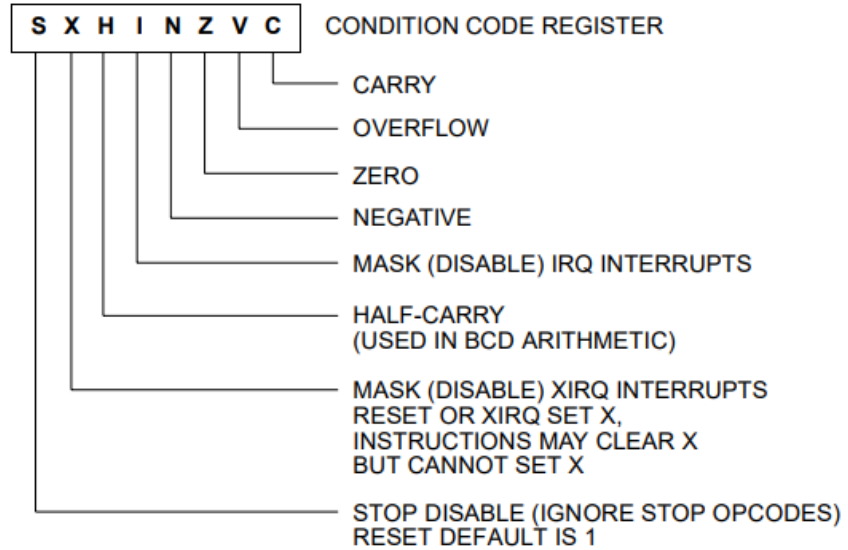
7	A	0	7	B	0	8-BIT ACCUMULATORS A AND B OR
15	D				0	16-BIT DOUBLE ACCUMULATOR D

15	X	0	INDEX REGISTER X
----	----------	---	------------------

15	Y	0	INDEX REGISTER Y
----	----------	---	------------------

15	SP	0	STACK POINTER
----	-----------	---	---------------

15	PC	0	PROGRAM COUNTER
----	-----------	---	-----------------



Lab1 :Introduction to the MC9S12 microcontroller

Task1:

Open the schematic diagram 'CSMB12_SCH_A1_0_0' located in the 'Datasheets / Board_diagrams' folder .

- Locate the MCU and check the number of data ports.
- Locate the external crystal clock .
- Follow the connections and find the :
 - LEDs
 - Switches
 - Push buttons
 - Potentiometer
 - Phototransistor
- What is the role of connector USER ?

Task2:

Codewarrior

Address of the beginning of the program in Flash EEPROM

This space is reserved to declare the program variables, which are saved in the RAM starting at @\$1000.

The program starts after ORG ROMStart, and the first instruction is saved at @\$4000.

This is an example program, to be deleted in order to write our own program.

Do not modify

```

ROMStart EQU $4000 ; absolute address to place my code/constant data

; variable/data section
; Insert here your data definition
Counter DS.W 1
FiboRes DS.W 1

; code section
ORG ROMStart

Entry:
_Startup:
    LDS #FRAMEEnd+1 ; initialize the stack pointer
    CLI              ; enable interrupts

mainLoop:
    LDX #1           ; X contains counter
counterLoop:
    STX Counter      ; update global
    BSR CalcFibo     ; store result
    STW FiboRes      ; store result
    LDX Counter
    CPX #24          ; larger values cause overflow
    BNE counterLoop ; restart
    BSR mainLoop

CalcFibo: ; Function to calculate fibonacci numbers. Argument is in X.
    LDY #000        ; second last
    LDD #001        ; last
    DBEQ X,FiboDone ; loop once more (if X was 1, were done already)
FiboLoop:
    LEAV D,Y        ; overwrite second last with new value
    EXG D,X          ; exchange them -> order is correct again
    DBEQ X,FiboLoop
FiboDone:
    RTS

;***** Interrupt Vectors *****
;***** Do not modify *****
ORG $FFFE
DC.W Entry ; Reset Vector
  
```

Write the following codes and test the functionality of each one :
(pay attention to the used addressing mode).

- a) `ldaa $1000 ;direct addressing mode`
`adda $1002`
`staa $1004`
- b) `ldaa #$02 ;immediate addressing mode`
`ldab #$03`
`addd #$0102`
`std $1020`
- c) `movw #$10,$1006`
`movw #$1234,$1008`
`ldx $1007`
- d) `ldab #$04`
`ldaa $1000`

again `adda #3`
`dec b ;inherent addressing mode`
`bne again ;relative short branch`
`staa $1002`
`ldd $1002`

- e) Define the variable table in the 'insert your data definition' section before writing the following code :

`table dc.b 1,2,3,4,5,6,7,8 ; define constant byte`

```
ldx table
ldx #table
ldaa 0,x ;indexed addressing mode
ldab 1,x
ldd 2,x
ldaa 1,x+
ldaa 2,x+
ldaa 1,+x
ldab -2,x
```

ldab #\$02
ldaa b,X

Task3:

- a) Write a program that will multiply the content of PORTA by 5 without using the multiplications instructions .

Note that the @PORTA = \$0000 .

- b) Write a program that will test the content of PORTA and :
- i. write the value #\$F0 at \$1010 if PORTA = 1 .
 - ii. write the value #\$00 at \$1010 if PORTA= 0 .
- c) Clear the 5th bit of PORTA in two ways.
- d) Set the bit 7th bit of PORTA in two ways.
